Лабораторна робота № 3–ЕС
Правила в середовищі CLIPS

Мета роботи – вивчити особливості реалізації правил у середовищі CLIPS для утворення БЗ.

Теоретичні відомості

Евристичний механізм подання знань у CLIPS реалізується за допомогою ***правил,*** тобто вони служать для подання евристик або так званих "емпіричних правил", які визначають набір дій, що виконуються при виникненні деякої ситуації. Правила складаються з *передумов (умов)* і *дії:*

ЯКЩО умова_1 і ... і умова_N виконуються, ТО
ВИКОНАТИ дію_1 і ... і дію_N

Список умов називається *лівою частиною правила* (Left-Hand Side або LHS), список дій – *правою частиною правила* (Right-Hand Side або RHS). Можливість застосувати конкретне правило визначається тим, чи виконуються умови, які сформульовані в його лівій частині. Виконання або невиконання умов визначається в момент їх зіставлення з фактами, які утворюють базу даних. Якщо кожна умова в лівій частині правила знаходить себе серед фактів, відбувається активізація правила і виконуються ВСІ дії, записані в його правій частині. В іншому випадку правило не активізується.

Работа правила дуже нагадує умовний оператор *if-then*, присутній у багатьох процедурних мовах програмування. Принципова різниця полягає у тому, що оператор *if-then* виконається у будь-якому випадку, коли до нього дійде черга в програмі. Що стосується правила, то інтерпретатор CLIPS ще "подумає", виконувати його чи ні. Так, при старті програми, що містить безліч фактів і правил, інтерпретатор CLIPS запускає машину логічного висновку, яка з'ясовує, які з правил можна активізувати. Це виконується циклічно, причому кожен цикл складається з трьох кроків:

- зіставлення фактів і правил;
- вибір правила, що підлягає активізації;
- виконання дій, запропонованих правилом.

Таким чином, правила, які взаємодіють з БД у вигляді фактів, вносять до неї функціональність і утворюють разом з нею БЗ. У CLIPS така БД може являти собою деяку предметну область, вихідний або поточний стан будь-якої проблеми, може моделювати в просторі або в часі поведінку будь-якої системи або будь-якої сутності, яку можна описати за допомогою безлічі записів у вигляді списків.

Для створення правила використовується конструктор ***defrule*,** який має такий *синтаксис:*

```
(defrule <Ім'я_правила>
    [необов'язковий коментар]
    [необов'язкова об'ява]
    (умова_1)
    (умова_2)
    ......
    (умова_M)
=>
    (дія_1)
    (дія_2)
    ......
    (дія_N))
```

Звернить увагу на те, що ліва частина правила відділяється від правої комбінацією символів **"=>",** а кількість умов і дій у правилі в загальному випадку не збігається.

Ім'я правила повинно бути значенням типу symbol і не може бути зарезервованим словом мови CLIPS. У разі повторного оголошення правила з однаковим ім'ям старе правило буде видалене, навіть якщо нове правило не можна буде додати внаслідок синтаксичної помилки або з будь-яких інших причин.

Коментар є необов'язковим – зазвичай у ньому описують призначення правила. Коментар буде значенням типу string, значення коментаря зберігається разом з правилом.

Передумова або *ліва частина правила* задається набором умовних елементів (conditional elements) – УЕ (CEs), що застосовуються до деяких *зразків.* Заданий набір зразків використовується системою для зіставлення з наявними фактами і об'єктами. Усі умови в лівій частині правила об'єднуються за допомогою неявного логічного оператора *and.* Якщо в лівій частині правила не зазначений жоден УЕ, CLIPS автоматично підставляє умову-зразок *initial-fact*. Таким чином, правило активізується кожного разу при появі в БЗ факту *initial-fact*.

Наслідок або *права частина правила* містить список дій, які виконуються при активізації правила механізмом логічного виводу. Дії правила виконуються послідовно, але тоді і тільки тоді, коли всі умовні елементи в лівій частині цього правила задоволені.

Розглянемо приклад програми, який показує кількість учасників деякої конференції, що приїхали з різних міст. Потрібно визначити кількість представників від кожного міста.

Для вирішення поставленого завдання нам необхідно виконати деяку послідовність дій, а саме:

1. Створити БД учасників, в якій на кожного учасника виділяється один запис (факт), що містить прізвище учасника і місто, з якого учасник прибув. Використовуючи будь-який текстовий редактор, створимо і збережемо БД у вигляді текстового файлу з ім'ям, що повторює ім'я БД **(conf.txt).** Вміст фактів бази даних може бути, наприклад, таким:

```
(deffacts conf
    (conf Alejnov Odessa)
    (conf Ladak Odessa)
    (conf Slobodjanjuk Lvov)
    (conf Klitka Lvov)
    (conf Bojko Kiev)
    (conf Pustovit Odessa)
    (conf Spokojnij Odessa)
    ( conf Shamis Odessa)
    (conf Lobovko Kiev)
    (conf Zadorozhna Lvov)
    (conf Javorskij Lvov))
```

2. У вікні редактора CLIPS створити і зберегти файл з ім'ям **conference.clp** (стандартне розширення CLIPS), який реалізує алгоритм (у вигляді правил) підрахунку кількості представників від кожного міста на конференції: для кожного міста задаємо лічильник і послідовно переглядаємо списки в записах файлу conf.txt. Якщо в запису поле списку має значення Kiev, то вміст відповідного лічильника збільшуємо на одиницю. Для інших міст – аналогічно. На мові CLIPS зазначений алгоритм може бути реалізований таким чином:

```
(defglobal ?*odessa* = 0)
(defglobal ?*kiev* = 0)
(defglobal ?*lvov* = 0)

(defrule begin
(initial-fact)
=>
(printout t crlf “Members of conference” crlf)

(defrule odessa
(conf ? Odessa)
=>
(bind ?*odessa* (+ ?*odessa* 1)))

(defrule kiev
(conf ? Kiev)
```

```
=>
(bind ?*kiev* (+ ?*kiev* 1)))

(defrule lvov
(conf ? Lvov)
=>
(bind ?*lvov* (+ ?*lvov* 1)))

(defrule result
(declare (salience –1))
(initial-fact)
=>
(printout t «from Odessa: « ?*odessa* crlf)
(printout t «from Kiev: « ?*kiev* crlf)
(printout t «from Lvov: « ?*lvov* crlf))
```

У перших трьох рядках програми за допомогою конструктора ***defglobal*** оголошуються три *глобальні змінні*: ?*odessa*, ?*kiev* і ?*lvov*. Ці змінні є лічильниками. У CLIPS змінна може бути і локальною – але тоді вона пов'язується тільки з тим правилом, в якому вона оголошується.

Далі йде правило з ім'ям begin, яке активізується незалежно від фактів у файлі conf.txt і присутнє у програмі тільки з однією метою – вивести заголовок. Для цього в його правій частині викликається вбудована функція ***printout*** з ключем ***t***, що виводить на стандартний пристрій виведення (монітор) заголовок, взятий у лапки. Комбінація символів ***crlf*** є аналогом *endl* в C++ і служить для перекладу курсору на наступний рядок. Наступні три правила з іменами *odessa, kiev* і *lvov* можна назвати ядром програми. У них проводиться підрахунок кількості учасників відповідно з Одеси, Києва і Львова.

Розглянемо правило *lvov*. Воно активізується у тому випадку, коли в БД знаходиться факт *(conf ? Lvov).* Символ **"?"** у другому полі цього списку означає символ універсальної підстановки і замінює собою будь-яке прізвище. Звідси випливає, що правило *lvov* активізується стільки разів, скільки разів факт *(conf ? Lvov)* присутній у БД. При цьому стільки ж разів виконаються дії, що містяться в правій частині правила. Вбудована функція ***bind*** – аналог оператора присвоювання. Отже, вміст змінної *?*lvov** збільшується на одиницю і результат зберігається в цій же змінній. Аналогічно працюють правила *odessa* і *kiev*.

У CLIPS існує декілька стратегій черговості виконання правил, а самі правила можуть мати пріоритет, який задається вбудованою функцією ***declare*** з параметром ***salience*** *(особливість).* Цей параметр може приймати цілочислові значення від –10000 до +10000. За замовчуванням для всіх правил величина ***salience*** дорівнює нулю.

Якщо у правилі *result* не вказати пріоритет, воно буде конфліктувати з правилом *begin* за черговість виконання, оскільки у цих правил однакова ліва частина. Для усунення конфлікту в правилі result пріоритет вказано явно і зі знаком мінус, у зв'язку з чим це правило виконується останнім.

3. Виконати розроблений алгоритм, що означає: командою ***(load ім'я_файлу)*** завантажити в інтерпретатор CLIPS файли *conf.txt* і *confrence.clp*, командами ***(reset)*** і ***(run)*** запустити програму *confrence.clp* на виконання:

```
CLIPS> (load conf.txt)
.........
TRUE
CLIPS> (load confrence.clp)
.........
TRUE
CLIPS> (reset)
CLIPS> (run)

Members of conference

from Odessa: 5
from Kiev: 2
from Lvov: 4
CLIPS>
```

Повідомлення інтерпретатора TRUE означає, що у файлі немає синтаксичних помилок і команда завантаження виконана коректно. Трикрапками подані інші повідомлення інтерпретатора, які в даному випадку опущені.

Порядок виконання роботи

1. Реалізувати наведений приклад у середовищі CLIPS.
2. Створити алгоритм і програму вирішення такого завдання у середовищі CLIPS:

Необхідно підрахувати і вивести на екран кількість студентів, що здали іспит з оцінками "A", "B" і "C" відповідно. Кількість студентів – не менше 10 осіб. Завантаження даних на виконання здійснити з файлів, використовуючи команду load.

Звіт про виконання роботи

Звіт про виконання лабораторної роботи № 3–ЕС має містити:

1. Титульний аркуш.
2. Програмний код, що складається з необхідного набору команд і функцій, а також результатів їх виконання для кожного пункту даної лабораторної роботи.
3. Екранні форми, що показують результати виконання кожного пункту даної лабораторної роботи.
4. Висновки.